

Compile-time Object-Relational Mapping

библиотека на C++

Дипломна работа

Алекс Цветанов, фак. № 121222225

Научен ръководител: проф. д-р инж. Румен Трифонов

Технически университет -- София
Факултет „Компютърни системи и технологии“
Компютърно и софтуерно инженерство

2026

Проблемът със заявките, проверявани при изпълнение

Традиционен подход:

- Заявките се описват като **низове**
- Валидиране едва при изпълнение
- Грешни имена на полета → срыв при изпълнение
- Несъвместими типове → късно откриване
- Смяна на хранилището → **пълно пренаписване**

Изследователски въпрос:

Може ли компилаторът да поеме ролята на първи валидатор на заявките?

При изпълнение	При компилация
Низови заявки	<code>constexpr</code> междинно представяне
Грешка при изпълнение	Грешка при компиляция
Късно откриване	Ранно откриване

Цел

Компилаторът поема ролята на първи валидатор на заявките.

Съществуващи решения и позиция на разработката

Решение	Подход	Ограничение
SOI ODB	при изпълнение кодогенерация	низови заявки външен генератор
sqlpp11 Hiberlite	вграден DSL ORM	главно за SQL само за SQLite

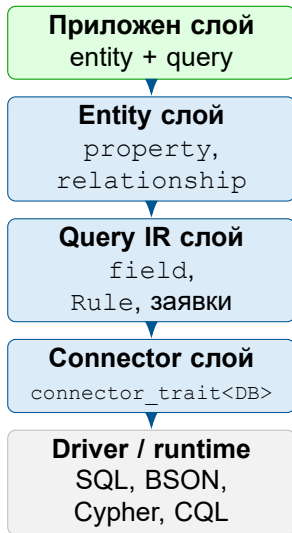
Цел на разработката

- `constexpr` междинно представяне
- Договор на конектора и способности
- 7 системи + корутинен `async` слой

Основна теза

Един C++ модел, валидиран при компилация, разширяем към много системи и практически приложим при асинхронни натоварвания.

Слоеста архитектура на библиотеката



Ключови архитектурни принципи:

- Без виртуален интерфейс --- признаков (trait-based) договор
- `constexpr` междинно представяне на заявката --- ранна валидация
- Маркери за способности (capability tags) ограничават неподдържаните операции
- Смяна на системата за съхранение = смяна само на типа

Централен инвариант

Приложният код не комуникира директно с драйвера --- работи само с C++ типове.

Минимален compile-time пример

```
struct User {  
    orm::property<int, "id"> id;  
    orm::property<std::string, "name">  
        name;  
};  
  
static constexpr auto q =  
    orm::select(  
        orm::field<&User::id>,  
        orm::field<&User::name>)  
    .where(orm::field<&User::id>  
        == orm::Placeholder<int>{{}});
```

- `property<T, "name">` носи тип и compile-time име на колона
- Заявката се изгражда като `constexpr IR`, а не като низ
- Грешно поле или неподдържана операция се открива рано

Ключова идея

Компиляторът поема ролята на първи валидатор на заявката.

Договор на конектора и механизъм на способностите

Минимален договор на конектора:

```
template<>
struct orm::connector_trait<MyDB> {
    template<typename T> struct
        wire_type;
    using cursor_type = ...;

    using supports_transactions =
        true_type;

    static auto execute(
        MyDB& db, auto query,
        auto&&... params);
};
```

- `is_connector<DB>` валидира интерфейса при компилация
- Маркерите за способности ограничават неподдържаните операции
- Смяната на системата за съхранение е смяна на типа, а не на интерфейса за заявки

Демонстрирани системи за съхранение

SQLite, PostgreSQL, MySQL, MongoDB, Redis, Neo4j, Cassandra

Един модел, много системи за съхранение

Едно междинно представяне → различни системи:

Система	Материализация
SQLite	WHERE id = ?
PostgreSQL	WHERE id = \$1
MySQL	WHERE id = ?
MongoDB	BSON filter
Redis	GET/SET/DEL
Neo4j	Cypher MATCH...WHERE
Cassandra	CQL SELECT...FROM

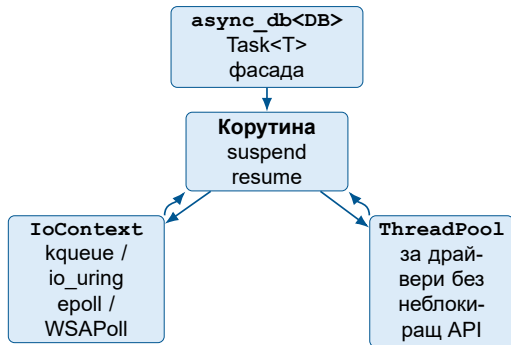
Абстракция на връзките:

embed	MongoDB вложен документ
reference	SQL foreign key / JOIN Redis: key prefix Neo4j: edge

Принцип

C++ моделът е единственият *логически* носител на схемата. Системата за съхранение определя физическата материализация.

Асинхронна архитектура на изпълнението



- Работната нишка не блокира; корутината се преустановява и продължава при готовност
- **Неблокиращ път:** PostgreSQL, MySQL/MariaDB, Cassandra, Redis
- **Обединителен път:** SQLite, MongoDB, Neo4j

Ключов извод

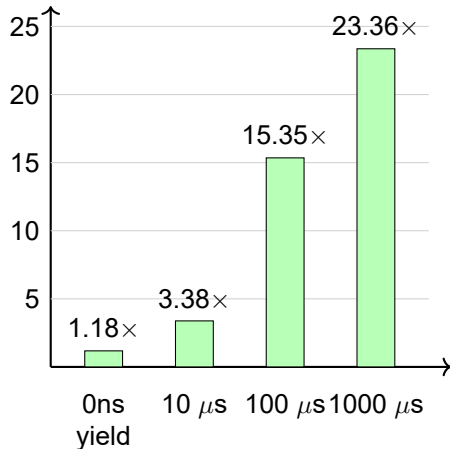
Асинхронният слой е втора архитектурна ос, а не обвивка за удобство.

Доказателствена база

271 теста --- 100% преминаване
при компилация + единични +
интеграционни + срещу живи системи

- Междинно представяне, способности, миграции, нишкова безопасност
- Асинхронна инфраструктура: `Task<T>`, `ThreadPool`, `IoContext`
- Емпирични измервания в `bench_async.cpp`

ускорение async/sync



4 нишки, 100 задачи --- ефектът расте с изчакването

Основни резултати и извод

- 1 ORM модел с проверка при компилация в стандартен C++23, без външна кодогенерация
- 2 Един C++ обектен модел за 7 системи за съхранение
- 3 Формален договор на конектора и разширяемост чрез способности
- 4 Асинхронен слой с измерима полза при натоварвания, ограничени от вход-изход
- 5 Многостепенна верификация: 271 теста + проверки срещу живи системи + емпирични измервания

Основен извод

Освен ORM с проверка при изпълнение (удобство, но късно откриване на грешки) и прекия достъп през драйвера (контрол, но без абстракция), е възможна **трета алтернатива**: договор на конектора, валидиран от компилатора, плюс корутинно изпълнение.

Бъдещо развитие:

- Отражение (reflection) от C++26
- Измервания срещу мрежови сървъри

Благодаря за вниманието!

Въпроси?